Homework 2: K-Nearest Neighbors

Make sure you have these libraries installed. Run the cell below to check and install them if needed.

In [452…

```
# Check and install required libraries
%pip install numpy matplotlib scikit-learn --quiet
```

```
Note: you may need to restart the kernel to use updated packages.
```

In [453…

```
# Basic imports used in several parts (you will need more imports later)
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# Initial dataset for parts 1 and 2
X, y = make_blobs(n_samples=500, centers=2, n_features=2, random_state=6, cluster_s

# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta
```

Part 1: K-Nearest Neighbors in Numpy

In [454…

```
# Follow the HW instructions to make your own classifier in Numpy, evaluate its acc

# This function executes the core logic of KNN. Namely, it finds the k nearest
# neighbors to point, and then finds and returns the most common label among those
# Write a function that takes in a point, some data, and a parameter k,
# and returns the k closest points to the input
def predict_y(point, x_data, y_data, k):
  # Implement this function, using the tips below.
  # There are many acceptable ways of doing this, but a straightforward one
  # is using a sequence of optimization loops. This will be inefficient, but plenty
  # fast for our small datasets.

  labels = []

  # TODO: First, make copies of your input data so you don't modify them.
  copy_x_data = list(x_data)
  copy_y_data = list(y_data)
  point_x = point[0]
  point_y = point[1]

  # TODO: Next, find the closest value in x_data to point, and add its label (in y_
  # the list labels. Then, remove these points from x_data and y_data.

  for _ in range(k):
    distance = ((point_x - copy_x_data[0][0])**2 + (point_y - copy_x_data[0][1])**2
```

```
    index = 0
    # Find the least distance between two points from the given data.
    for i in range(1, len(copy_x_data)):
      new_distance = ((point_x - copy_x_data[i][0])**2 + (point_y - copy_x_data[i][
      if new_distance < distance:
        distance = new_distance
        index = i

    copy_x_data.pop(index)

    labels.append(copy_y_data[index])

    copy_y_data.pop(index)

  # TODO: Repeat the above k times (using a loop)

  # TODO: Return the majority element in labels.
  count_0 = 0
  count_1 = 0
  for num in labels:
    if num == 0:
      count_0 += 1
    else:
      count_1 += 1
  if count_0 >= count_1:
    return 0
  else:
    return 1
```

In [455…

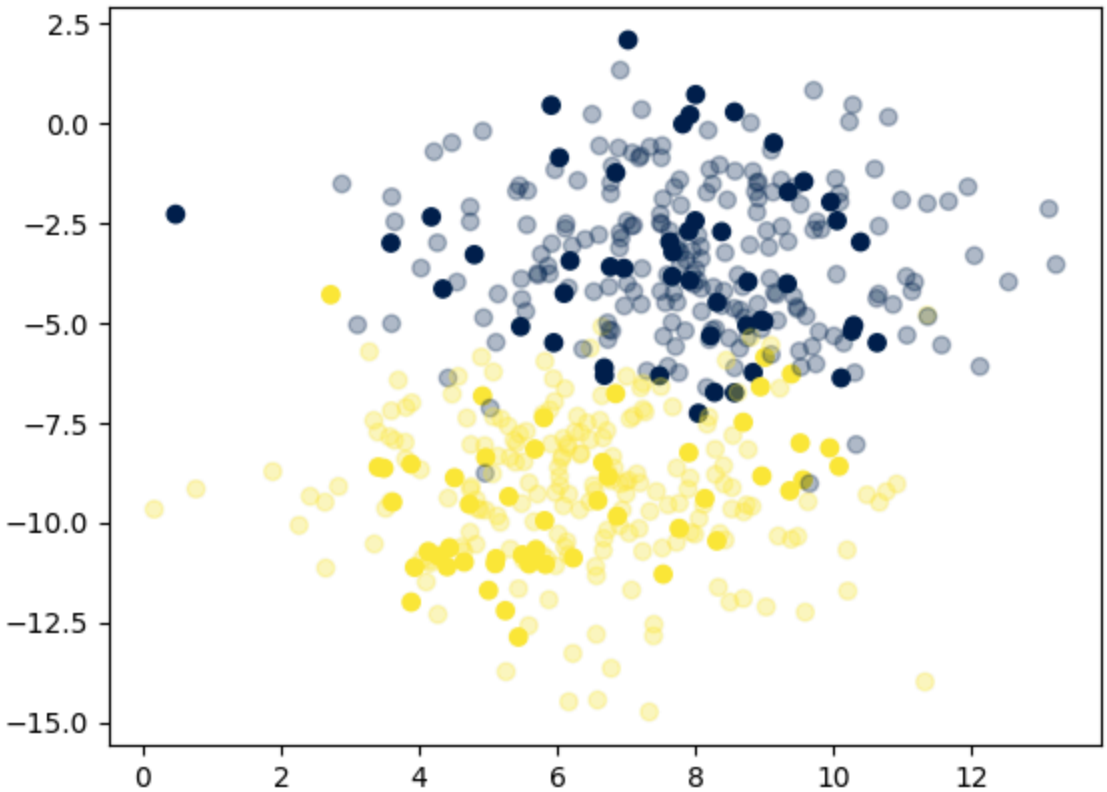
```
# TODO: Using your function above, predict the labels of all points in the testing
correct_prediction_count = 0
predicted_labels = []
for i in range(len(X_test)):
    correct_label = y_test[i]
    prediction = predict_y(X_test[i], X_train, y_train, 20)
    predicted_labels.append(prediction)
    if (correct_label == prediction):
        correct_prediction_count += 1

# TODO: Evaluate and print the accuracy of your classifier using the testing data (
# (if your accuracy is below 0.9, you have probably made some sort of mistake)
print(correct_prediction_count / len(X_test))
```

```
0.9
```

In [456…

```
# TODO: Create a scatter plot to visualize your results. Make sure to follow the in
# document, namely you should use point color to represent their predicted labels,
# values to distinguish between training and test points.
actual_labels = list(y_test)

# Drawing the scatterplot for the test data.
x_coordinates = []
for x in X_test:
    x_coordinates.append(x[0])

y_coordinates = []
```

```
for y in X_test:
    y_coordinates.append(y[1])

plt.scatter(x_coordinates, y_coordinates, c = y_test, label = "Test Set", cmap = "c

x_coordinates.clear()
y_coordinates.clear()

# Drawing the scatterplot for the training data.
for x in X_train:
    x_coordinates.append(x[0])

for y in X_train:
    y_coordinates.append(y[1])

plt.scatter(x_coordinates, y_coordinates, c = y_train, label = "Training Set", alph

plt.show()
```



Part 2: K-Nearest Neighbors in Sklearn

In [457…

```
# Do the same analysis as part 1, but using KNeighborsClassifier from sklearn.neigh
# Step 1. Build KNN
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors = 20)
knn.fit(X_train, y_train)
```

Out[457…

```
KNeighborsClassifier
▶ Parameters
```

In [458…

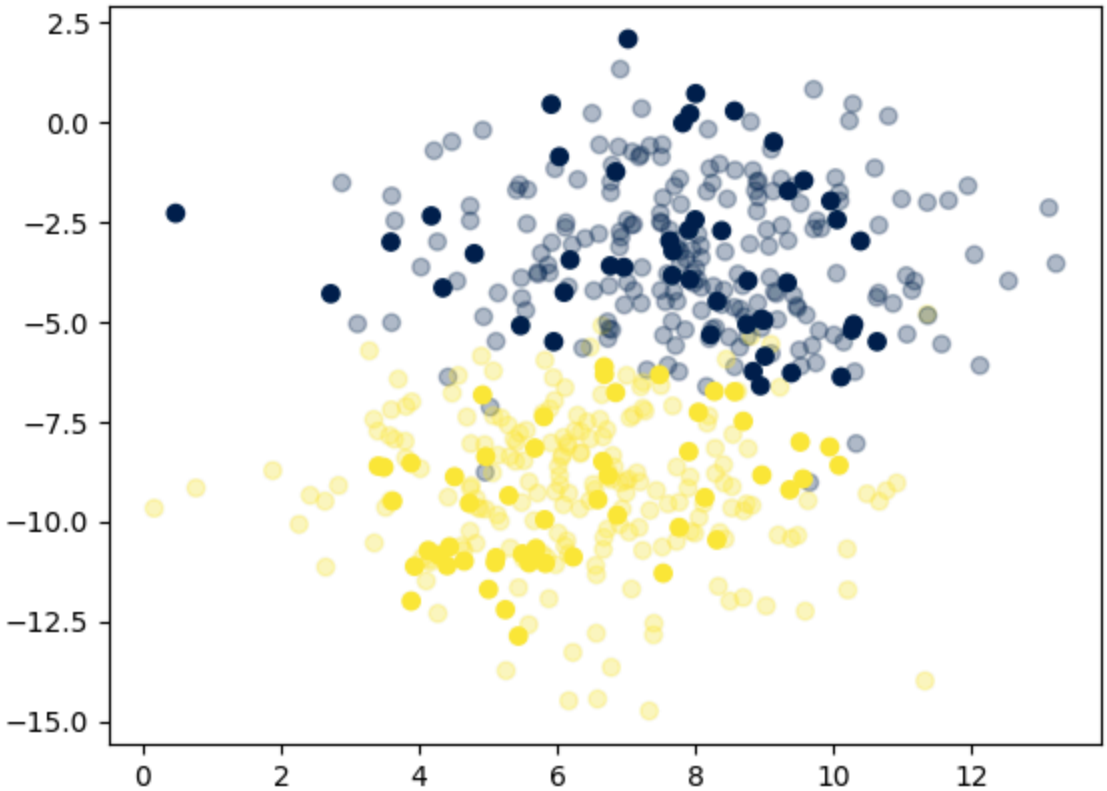
```
# Step 2. Calculate the Accuracy (>0.9)

prediction_list = []
copy_x = list(X_test)
for point in copy_x:
    prediction = (knn.predict([point]))
    prediction_list.append(prediction)

count = 0
for i in range(len(y_test)):
    if (prediction_list[i] == y_test[i]):
        count += 1

print(knn.score(X_test, y_test))
```

```
0.9
```

In [459…

```
# Step 3. Plot

# Drawing the scatterplot for the test data.
x_coordinates = []
y_coordinates = []

# Drawing the scatterplot for the training data.
for x in X_train:
    x_coordinates.append(x[0])

for y in X_train:
    y_coordinates.append(y[1])

# Here the training set is drawn with a faint color.
plt.scatter(x_coordinates, y_coordinates, c = y_train, label = "Training Set", alph

# Now draw the predicted data.
x_coordinates.clear()
y_coordinates.clear()

for x in X_test:
    x_coordinates.append(x[0])

for y in X_test:
    y_coordinates.append(y[1])

plt.scatter(x_coordinates, y_coordinates, c = prediction_list, label = "Prediction

plt.show()
```

```
# The faint points are the training points.
# The more brighter points are the predicted points.
# The blue points have the label 1.
# The yellow points have the label 0.
```



Part 3: Beyond a simple case

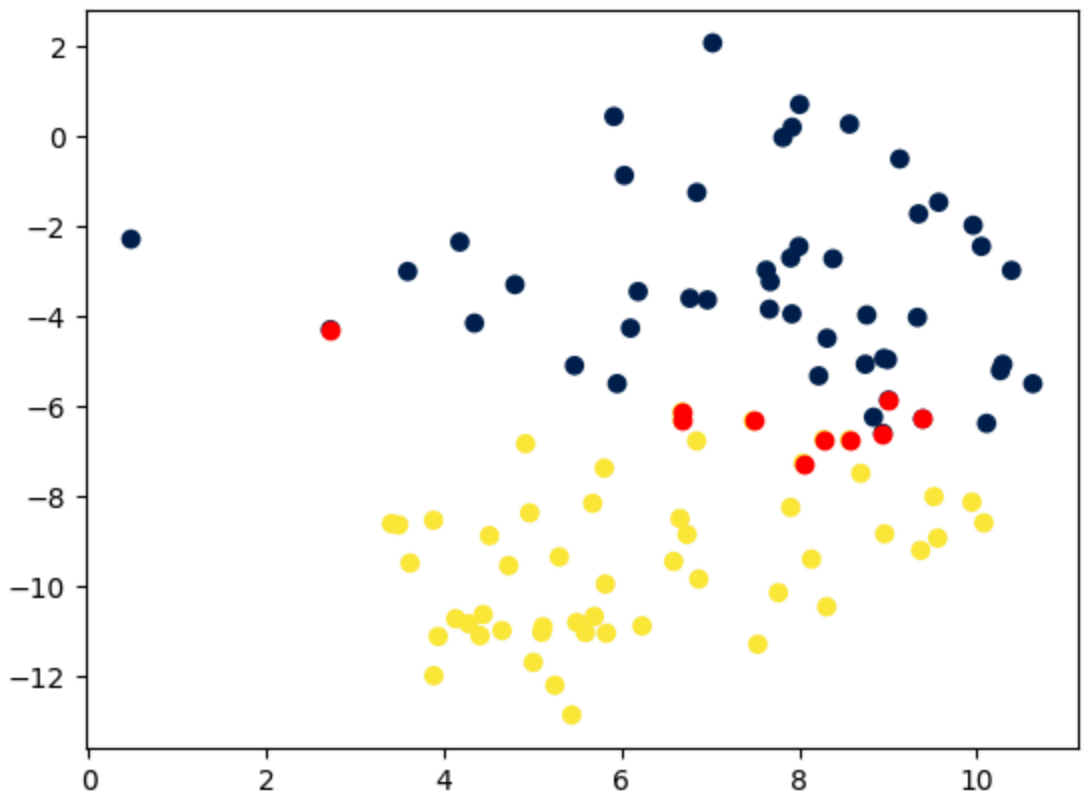
In real life, things are typically more complicated than the example we've been practicing with. For example, data can be messier, and you have to make several decisions about details of your algorithm. In this part, we will see several examples of this.

In [460…

```
# In Parts 1 and 2, your final scatter plot displays the predicted labels for each
# However, we cannot tell which predictions are correct and which are incorrect.
# Make another scatter plot displaying the prediction errors.
# That is, the color of each test point should depend on whether the prediction was
# (make sure to also show the training points, as before).
# TODO: Plot the figure

plt.scatter(x_coordinates, y_coordinates, c = prediction_list, label = "Prediction

x_coordinates.clear()
y_coordinates.clear()

for i in range(len(prediction_list)):
    if prediction_list[i] != y_test[i]:
        x_coordinates.append(X_test[i][0])
```

```
        y_coordinates.append(X_test[i][1])

plt.scatter(x_coordinates, y_coordinates, color = "red", label = "Inocrrect Predict
print("Incorrect prediction points are shown in red.")

plt.show()


# These are the predicted points.
# The red points are the incorrectly predicted points.
```

```
Incorrect prediction points are shown in red.
```

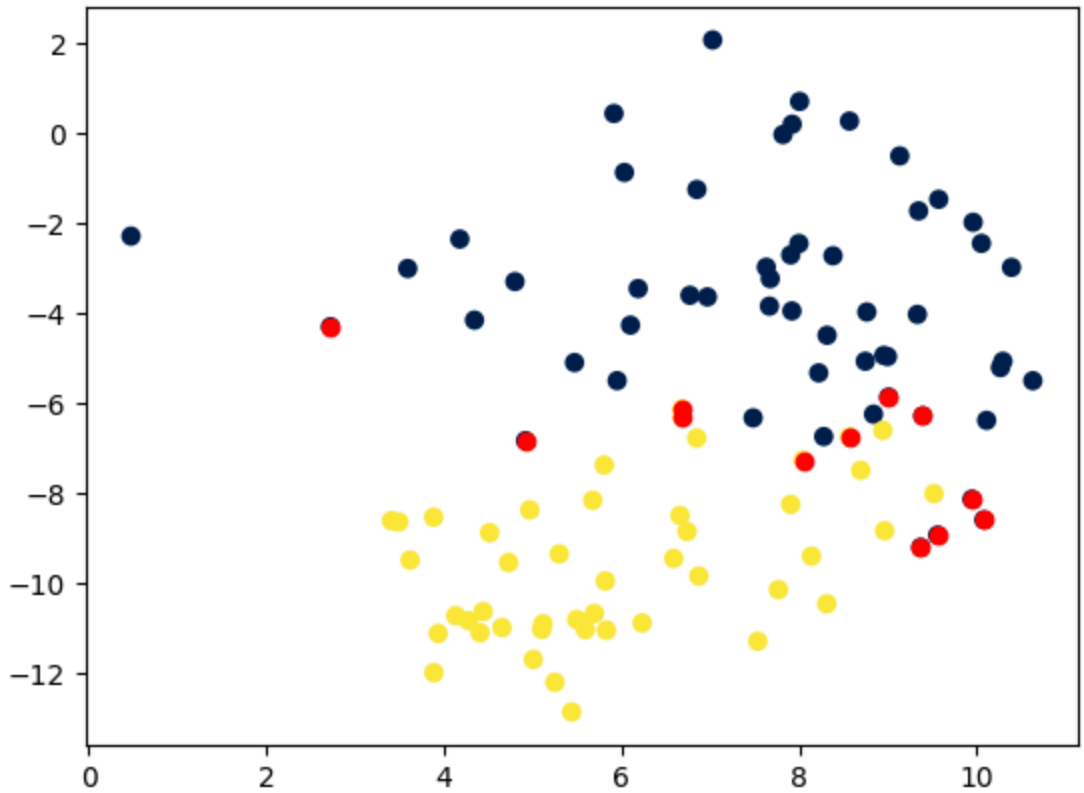


In [ ]:

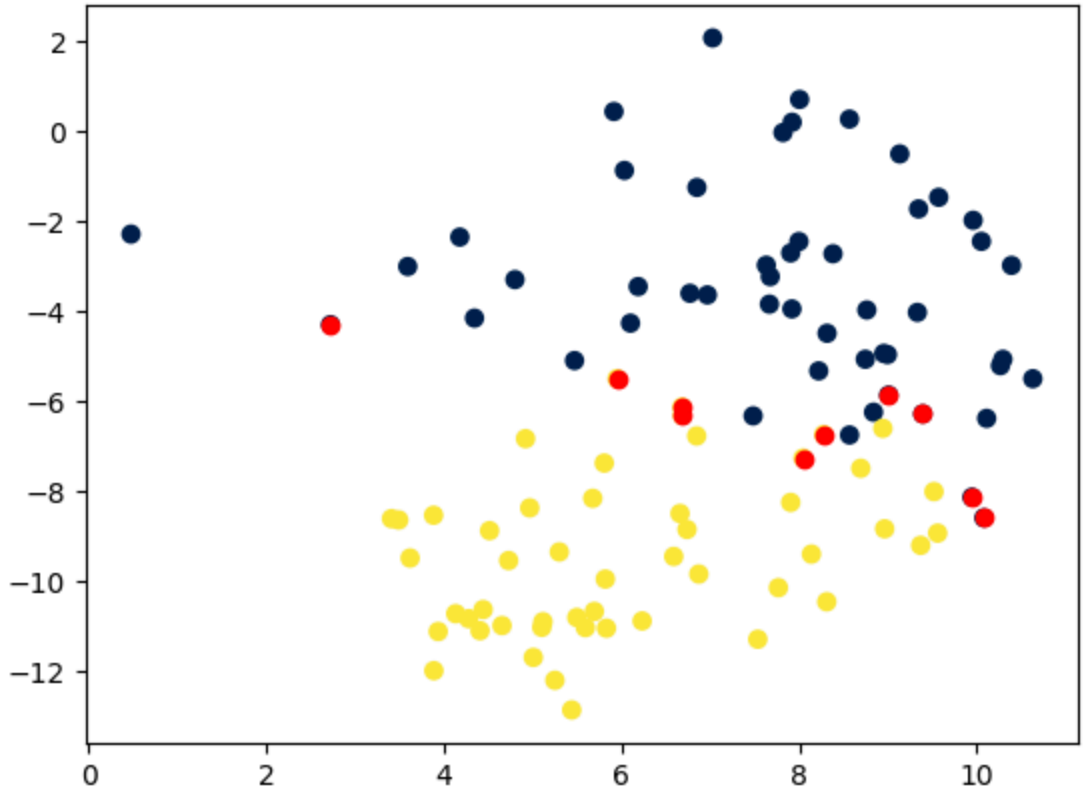
```
#Now, we will experiment with varying values of k (the main parameter of the model)
#Using the same data as above, train 3 KNN models with k=1,3, and 5.
# For each, display its accuracy, and show a scatter plot of prediction errors.
# Notice which particular points are correct/incorrect for which values of k, and h
# TODO: Experiment with different values of k (1,3, and 5) for the data above.

# Fitting the knn's
knn_1 = KNeighborsClassifier(n_neighbors = 1)
knn_1.fit(X_train, y_train)

knn_2 = KNeighborsClassifier(n_neighbors = 3)
knn_2.fit(X_train, y_train)

knn_3 = KNeighborsClassifier(n_neighbors = 5)
knn_3.fit(X_train, y_train)

knn_list = []
knn_list.append(knn_1)
knn_list.append(knn_2)
```

```
knn_list.append(knn_3)

knn_names = []
knn_names.append("knn_1")
knn_names.append("knn_2")
knn_names.append("knn_3")


# Running predictions on the three knn models and printing their accuracy.
for i in range(3):
    prediction_list = []
    for point in X_test:
        prediction = (knn_list[i].predict([point]))
        prediction_list.append(prediction)

    incorrect_predictions_coordinates_x = []
    incorrect_predictions_coordinates_y = []

    count = 0
    for j in range(len(y_test)):
        if (prediction_list[j] == y_test[j]):
            count += 1
        else:
            incorrect_predictions_coordinates_x.append(X_test[j][0])
            incorrect_predictions_coordinates_y.append(X_test[j][1])

    print(knn_list[i].score(X_test, y_test))
    print(f"Plot for: {knn_names[i]}")
    print("Incorrect prediction points are shown in red.")

    x_coordinates.clear()
    y_coordinates.clear()

    for point in X_test:
        x_coordinates.append(point[0])
        y_coordinates.append(point[1])

    # Drawing the plots.
    # First draw the predicted points.
    plt.scatter(x_coordinates, y_coordinates, c = prediction_list, label = "Predict

    # Then draw the incorrectly predicted points.
    plt.scatter(incorrect_predictions_coordinates_x, incorrect_predictions_coordina
    plt.title = f"Plot for: {knn_names[i]}"

    plt.show()



# The plots are shown below for each knn model's predicted points.
# The red points are the incorrectly predicted points.
```

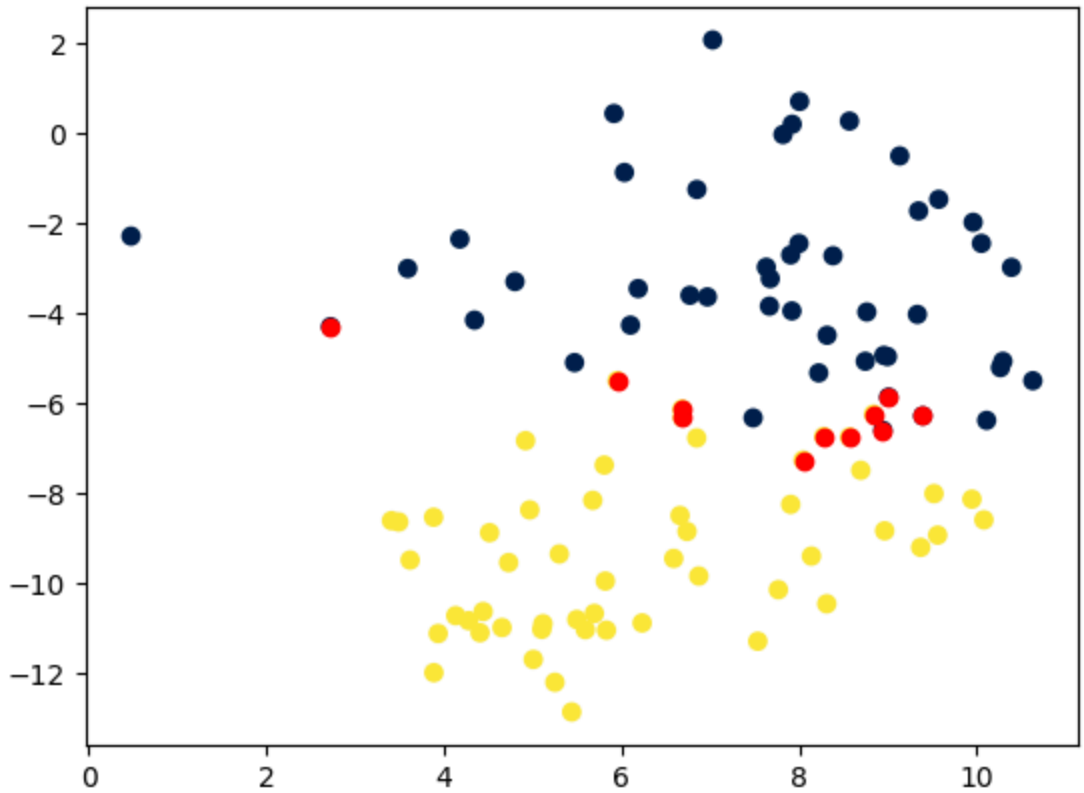
```
0.88
Plot for: knn_1
Incorrect prediction points are shown in red.
```




```
0.9
Plot for: knn_2
Incorrect prediction points are shown in red.
```




```
0.89
Plot for: knn_3
Incorrect prediction points are shown in red.
```



In [426…

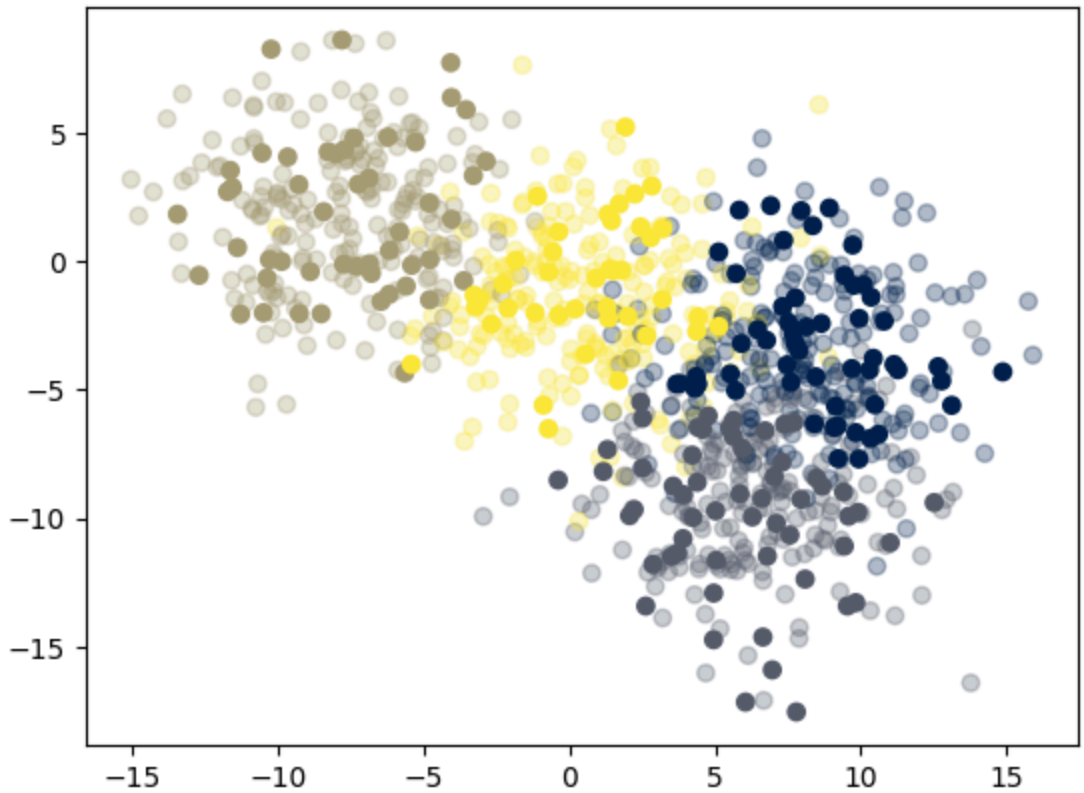
```
# Write a few sentences describing how and why the predictions change for different
# and justifying which choice of k you think seems best for this dataset.
# TODO: Write a few sentences justifying the best choice of k.

# Question: Write a few sentences justifying the best choice of k.

# Answer:
# Based on the few prediction plots, the best k number would be 3. I think 3 is doi
# prediction than than 1 and 5 k neighbors because of the way the points are distri
# points are clustered together while some are further away. But the way the label
# the points seem to be based on a line. I would say somewhere around Y = -6. And b
# knn model looks at the nearest neighbors and not the Y = -6 line the accuracy rat
# of low on all 3 knn models. k = 1 is just bad because I think it would be bad in
# it needs at least 2 or 3 neighbors for a majority vote. And 5 may be performing b
```

In [427…

```
# Now, we will try a more complex example. Following the pattern above, train anoth
# Try several parameters to try to achieve the best accuracy possible
# (you can try various values of k, but also note that KNeighborsClassifier support
# TODO: Try to train a good model for the following slightly more complicated datas
X, y = make_blobs(n_samples=1000, centers=4, n_features=2, random_state=6, cluster_

# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta

# Fitting the knn's
knn = KNeighborsClassifier(n_neighbors = 21)
knn.fit(X_train, y_train)

prediction_list = []
```

```
for point in X_test:
    prediction = (knn.predict([point]))
    prediction_list.append(prediction)

accuracy = knn.score(X_test, y_test)
print(f"The accuracy of this knn model with 4 types of labeled data is: {accuracy}"

# Drawing the scatterplot for the test data.
x_coordinates = []
y_coordinates = []

# Drawing the scatterplot for the training data.
for x in X_train:
    x_coordinates.append(x[0])

for y in X_train:
    y_coordinates.append(y[1])

# Here the training set is drawn with a faint color.
plt.scatter(x_coordinates, y_coordinates, c = y_train, label = "Training Set", alph

# Now draw the predicted data.
x_coordinates.clear()
y_coordinates.clear()

for x in X_test:
    x_coordinates.append(x[0])

for y in X_test:
    y_coordinates.append(y[1])

plt.scatter(x_coordinates, y_coordinates, c = prediction_list, label = "Prediction

plt.show()
```

```
The accuracy of this knn model with 4 types of labeled data is: 0.845
```

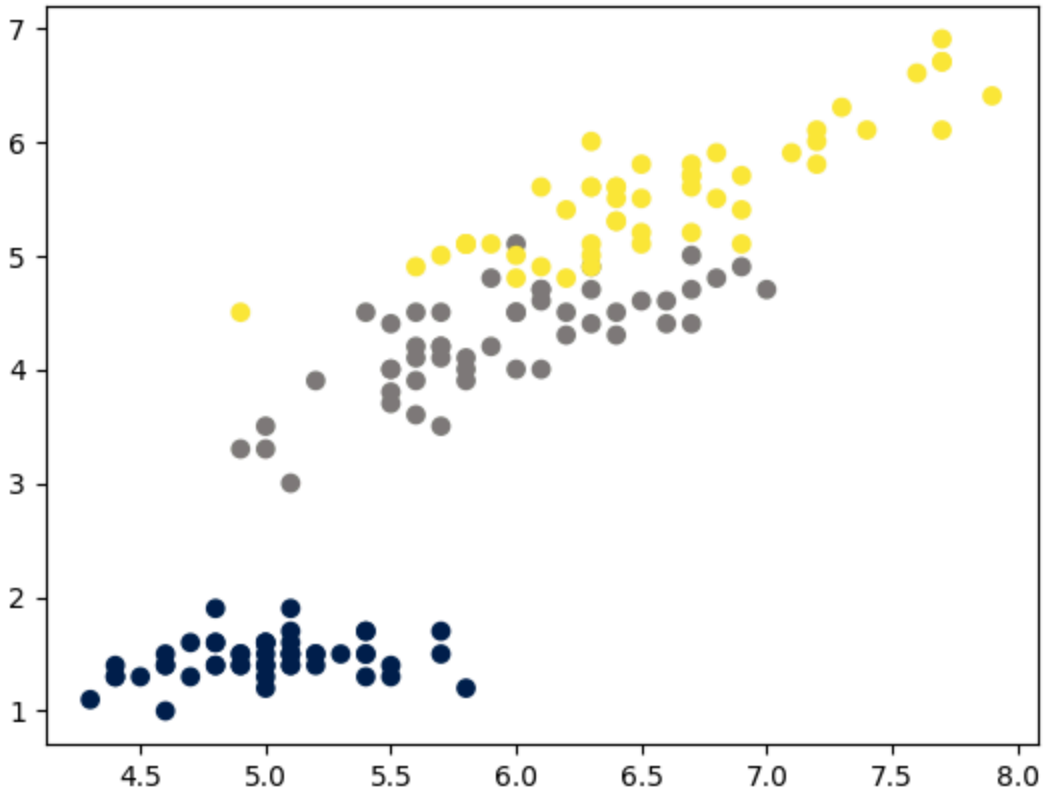


In [ ]:

Write a paragraph, with supporting code and plots, describing a good model for this dataset.

Part 4: A real dataset

In [ ]:
```
# TODO: try to train a *good* model for the following real world dataset.
# NOTE, this data is significantly messier than above. For example, the different f
# We will talk more about this in class, but for now, try to find a reasonable clas
data = load_iris()
X = data.data # type: ignore
y = data.target # type: ignore

# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta

# I also tried normalizing features but it was quite hard and I had to retract the
knn = KNeighborsClassifier(n_neighbors = 15, weights='distance')
knn.fit(X_train, y_train)

# Testing the model
accuracy = knn.score(X_test, y_test)
print(f"The accuracy of this knn model with 3 types of labeled data is: {accuracy}"

# Plotting it
plt.scatter(X[:, 0], X[:, 2], c = y, label = "Plot", cmap = "cividis")
```

```
name = "Sepal length vs Petal length"
plt.show()
```

```
The accuracy of this knn model with 3 types of labeled data is: 1.0
```